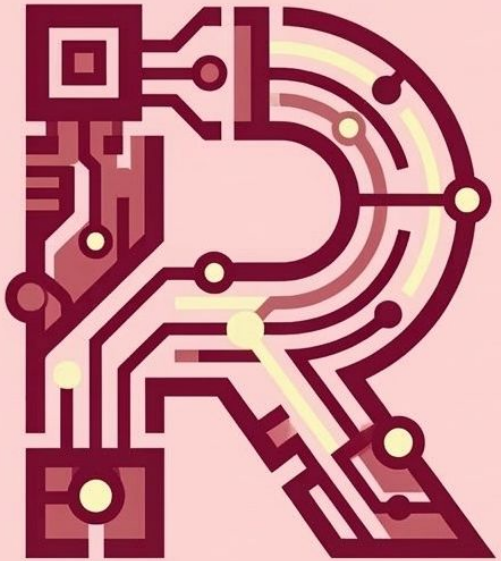


ReqPal

Your companion in the regulatory world

AI-Powered Requirements Management

From compliance documents and stakeholder input to validated, Jira-ready user stories.

For Product Managers · Compliance Teams · Stakeholders

The Challenge

Requirements management in regulated industries is broken

Especially true in small teams — law firms, consultancies, compliance units — where the product is a side project.

Lawyers are the essential SMEs. But billable hours don't leave room for classic req collection.

DOCS

Document overload

Regulations, specs, and PDFs pile up. Key requirements get buried or missed entirely by PMs who can't read everything.

COMM

Lost in translation

Stakeholder needs are captured in meetings and emails. Then forgotten, misinterpreted, or never turned into stories.

LOOP

Endless review cycles

Stories bounce between PM, stakeholders, and dev teams. No structured workflow means nothing ever gets properly signed off.

TIME

No time for workshops

Lawyers can't block 2 hours for a requirements session. Scheduling is a nightmare and the critical legal expertise never makes it into the backlog.

The Hidden Cost of Compliance Gaps

01

Manual traceability

Teams spend hours manually linking requirements to stories to test cases. One missed link = failed audit.

02

No stakeholder visibility

Stakeholders have no window into the system. They can't see, review or sign off on what's being built.

03

Context lost between tools

Documents live in SharePoint, stories in Jira, chat in Slack. Nothing connects. Requirements drift.

04

Export friction

Getting validated requirements into Jira means manual data entry. A single sprint's worth takes hours.

How ReqPal Solves It

One structured workflow — from raw documents to Jira-ready stories

1	2	3	4	5	6
Upload docs	**RAG search**	**Stakeholder chat**	**Story generation**	**Review & validate**	**Export to Jira**
PDF, DOCX, CSV, JSON	Instant Q&A over docs	AI extracts requirements	Grounded in docs + needs	Accept / reject stories	CSV or JSON, PM-approved

Challenge → ReqPal Solution

- Document overload → RAG engine with reranking instantly answers questions over any uploaded doc
- Lost stakeholder needs → AI chat assistant extracts and saves structured requirements automatically
- No time for workshops → Async stakeholder dashboards — each person contributes on their own schedule, no meeting required
- No traceability → Every story is grounded in source documents and stakeholder conversations
- Export friction → One-click CSV/JSON export of PM-validated stories, Jira-compatible

Key Features

RAG

RAG Engine

Local embeddings (BAAI/bge) + cross-encoder reranking. Answer mode or raw chunk search. No API cost for retrieval.

AI

AI Requirements Chat

Stakeholders describe needs in plain language. The AI extracts, structures and saves requirements automatically.

STORY

Story Generation

Stories grounded in both uploaded documents and stakeholder requirements. Full acceptance criteria included.

CHECK

Review Workflow

Token-based stakeholder links. Accept or reject each story. PM validation before export. Clear audit trail.

AI+

Cascading AI fallback

Groq -> Lightning AI -> LitServe -> Qwen local. The system never goes dark -- always a provider available.

EXPORT

Jira Export

CSV and JSON export with Jira-compatible fields. Only PM-validated stories are exported. One-click ready.

Architecture at a Glance

Layer	Technology
API	FastAPI
Persistence	SQLite
Vector Store	ChromaDB
Embeddings	BAAI/bge-small (CPU)
Reranker	ms-marco MiniLM (CPU)
Frontend	Vanilla JS, no build step

LLM Fallback Stack

1. **Groq (primary)**
   llama-3.3-70b · fast · free tier
2. **Lightning AI**
   fallback on rate-limit
3. **LitServe / Qwen**
   local GPU, opt-in
4. **OpenAI / Ollama**
   last resort

Two Roles. One Workflow.

Product Manager

- Upload documents (PDF, DOCX, CSV, JSON)
- Ask questions over docs via RAG search
- Create stakeholders & share dashboard links
- View all submitted requirements
- Manage, filter, and validate user stories
- Export PM-validated stories to Jira (CSV/JSON)

->

Stakeholder

- Access personal dashboard via token link (no login)
- Chat with AI assistant — describe requirements
- AI structures and saves requirements automatically
- Review generated user stories
- Accept or reject stories with a single click
- Export accepted stories as CSV or JSON

Story Lifecycle

Every story has a clear, auditable path from AI generation to Jira export

ai_generated

By: System

AI creates story from stakeholder requirements + docs

pending_review

By: Stakeholder

Story visible in stakeholder dashboard for review

accepted

By: Stakeholder

Stakeholder approves the story with one click

pm_validated

By: PM

PM reviews and validates the accepted story

exported

By: PM

Story exported to Jira-compatible CSV or JSON

Deployment Options

Railway (Docker)

Recommended for production

- CPU-only, no GPU required
- Auto-deploy on every push to main
- Persistent volume for DB & vector store
- Add GROQ_API_KEY + LIGHTNING_API_KEY as env vars

Lightning AI (GPU)

For local Qwen model

- L4 or A10G GPU instance (24 GB VRAM)
- Runs Qwen 2.5-7B via LitServe
- Groq + Lightning AI as cloud fallbacks
- Port forwarding to expose the app

Local (Dev)

Quick start

- Python 3.10+, free Groq API key
- No GPU needed with cloud providers
- uvicorn main:app --port 8001
- .env file for all configuration

Project Structure

`main.py`	FastAPI app — all API endpoints
`models.py`	Pydantic models & enums
`storage.py`	SQLite CRUD — all entities
`db_schema.py`	SQLite schema (13 tables)
`requirements.txt`	Full dependencies
`requirements-deploy.txt`	CPU-only (Docker/Railway)
`Dockerfile`	CPU-only container for Railway
`startup.sh`	Lightning AI startup script

backend/services/

`rag_service.py`	Chunking, embedding, ChromaDB retrieval
`reranker_service.py`	Cross-encoder reranking (lazy, CPU)
`llm_router.py`	Groq → Lightning → LitServe → Qwen → OpenAI → Ollama
`rag_llm_prompts.py`	All LLM prompt templates

static/

`index.html`	PM Dashboard (tabbed, responsive)
`dashboard.html`	Stakeholder Dashboard (responsive)

Future Development

ROADMAP

BPMN Process Integration

v2

Import BPMN diagrams and auto-extract process steps as draft requirements. Visual process map linked to stories.

Sprint Planning Assistant

v2

AI suggests story groupings into sprints based on priority, dependencies and team capacity. One-click sprint creation.

Stakeholder Matrix

v3

Map stakeholders by influence and interest. Auto-route requirements to relevant reviewers. Track sign-off per stakeholder.

Gap Analysis Engine

v3

Compare uploaded regulations against existing requirements. Highlight uncovered clauses and generate missing stories automatically.

Real-time Collaboration

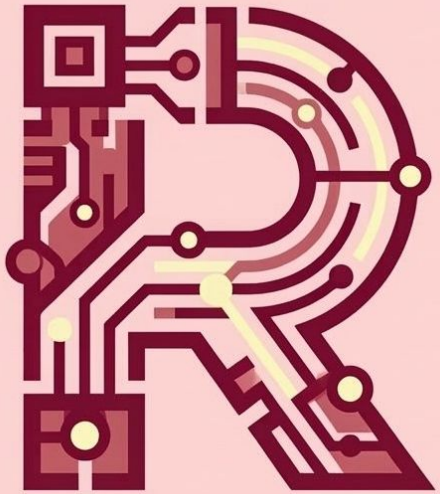
v4

WebSocket-powered live editing. PMs and stakeholders see changes instantly. Conflict resolution built in.

Change Impact Radar

v4

When a regulation is updated, AI flags affected requirements and stories. Shows blast radius before you commit a change.



ReqPal

Your companion in the regulatory world

Ready to get started?

GIT	Clone the repo and run locally in minutes
DEPLOY	Deploy on Railway with zero GPU requirements
KEY	Add your Groq + Lightning AI API keys — done
START	Upload your first compliance doc and start asking questions

github.com/Jo-llama/ReqPal